MC970 - Introdução à programação paralela
Relatório - OpenMP Tasks

Daniel de Sousa Cipriano RA: 233228

I. Descrição de sistema

Os exercícios foram executados em um sistema Linux baseado em Ubuntu com a seguinte configuração:

Componente	Descrição
Sistema operacional	Linux Mint 64 bits
Processador	AMD Ryzen 5 5500U com SMT
Núcleos	6 físicos, 12 lógicos
Clock	4.05 GHz
Memória RAM	16 GB DDR4

II. Ferramentas

As principais ferramentas utilizadas para desenvolvimento foram:

Hyperfine: gerador de benchmark. Recebe dois comandos e retorna dados comparativos de desempenho entre eles[1]. Foi utilizada a biblioteca `matplotlib` do Python, em conjunto com benchmarks em formato JSON do Hyperfine, para gerar dados e plotar gráficos de speedup por valores de entrada, eficiência por valores de entrada e tempo de execução serial/paralelo por valores de entrada.

OpenMP: O OpenMP(Open Multi-Processing) é uma API para programação de sistemas multiprocessados com memória compartilhada. A interface fornece diretivas de compilação, bibliotecas de funções e variáveis de ambiente que facilitam e padronizam a implementação de sistemas com multithreading[2].

Gperftools: O Gperftools é um conjunto de ferramentas, distribuído pela Google, para análise de performance em sistemas com multithreading[3]. O software fornece utilitários como o CPU profiler, para perfilamento de código com múltiplas opções de análise[4] e o Heap profiler que fornece análise de heap(alocação dinâmica, vazamentos de memória, etc)[5]. Para esse exercício, foi utilizado o CPU profiler em modo de análise gráfica. Essa configuração gera um grafo orientado de nós, cada nó representa uma função executada e cada aresta representa uma chamada de função, apontando da função origem para a função destino(incluindo funções de kernel). O CPU profiler trabalha coletando amostras periódicas do estado do programa em tempo de execução, registrando em que etapa o fluxo estava em cada medição. A saída gráfica exibe os seguintes campos:

- Nome do binário de execução
- Total samples: total de amostras periódicas coletadas
- Focusing on: amostras efetivamente utilizadas no grafo. Pode ser menos que o total caso sejam aplicados filtros de função, tempo ou ruído
- Dropped nodes with <= x abs(samples): indica descarte de nós que possuem menor amostragem do que o limite mínimo x(pode ser ajustado com - -nodefraction)
- Dropped edges with <= x samples: indica descarte de arestas que possuem menor amostragem do que o limite mínimo x(pode ser ajustado com - -edgefraction)

Dentro dos nós, estão as seguintes informações:

- Nome da função
- Número de amostras de tempo coletadas dentro da função
- Porcentagem do tempo total gasto na função
- Total de amostras cumulativas, incluindo as funções chamadas a partir desta, ou seja, as amostras foram gastas a partir

dessa função, mas não dentro dela diretamente

Por fim, as arestas representam uma chamada à função na direção especificada e junto de cada uma está o número de amostras em que a chamada ocorreu. Em alguns casos foram excluídas funções de kernel, para melhorar a visualização do grafo.

III. OpenMP Tasks: ZIP

A. Objetivo

O objetivo desse exercício foi paralelizar um código em C que implementa uma quebra de senha de arquivo zip, utilizando tasks do OpenMP. A diretiva `task` do OpenMP é utilizada dentro do bloco paralelo, delimitado pela diretiva `single`, permitindo que uma única thread, denominada como thread de controle (control thread), atribua tarefas às threads disponíveis no pool de threads, denominadas como threads de trabalho (worker threads). A versão serial do programa em `zip-serial.c`, executa um loop que testa senhas de 0 a 500000 uma a uma, até que a senha do arquivo zip seja encontrada.

B. Resultados

A versão paralelizada do código em `zip-parallel.c`, implementa dentro do loop de teste de senhas, um bloco OpenMP tasks que atribui à cada thread disponível, a tarefa de testar uma possibilidade de senha, de 0 até 500000. Threads livres automaticamente pegam um valor do loop e testam o retorno, salvando em um pipe o resultado do comando de descompressão de arquivo zip com uso de senha. Para controlar o fim da execução, foi criada a variável `password_found`, que é compartilhada entre threads. Ao encontrar a senha, a thread de execução entra em um bloco sincronizado com a diretiva `atomic write` e atribui 1 ao valor da variável de controle. Antes de iniciar a execução de uma parte do loop e durante a leitura do pipe, a variável de controle é verificada, se o valor dela está setado como 1, então a execução se encerra. O programa recebe a quantidade de threads que serão geradas no bloco paralelo e o caminho para o arquivo zip com senha; retorna a quantidade de tempo total de execução. O total de threads atribuído ao bloco paralelo por valor de senha foi de:

- 2 threads para quebra da senha 3333 no arquivo `file1.zip`
- 2 threads para quebra da senha 6666 no arquivo `file2.zip`
- 4 threads para quebra da senha 10000 no arquivo `file3.zip`
- 4 threads para quebra da senha 13333 no arquivo `file4.zip`
- 8 threads para quebra da senha 16666 no arquivo `file5.zip`

Foram gerados gráficos para análise de desempenho das versões do programa. Analisando o gráfico de speedup por valores de senha na Figura 1, vemos o crescimento do speedup com o aumento dos valores de senha. Esse comportamento reflete a compensação do custo de paralelização com o aumento da granularidade do problema. Analisando o gráfico de eficiência por valores de senha na Figura 2, vemos o decaimento da eficiência com o aumento no valor de senha. Esse comportamento ocorre devido à variação no número de threads utilizadas no bloco paralelo. O aumento no número de threads, adiciona custo de paralelização, devido à fatores como overhead de criação e gerenciamento de threads, custo de sincronização, contenção por recursos, etc. Esses fatores reduzem a eficiência da solução. Olhando para o gráfico de tempo de execução serial e paralela por valores de senhas na Figura 3, vemos o ganho de desempenho da versão paralela sobre a serial, com o tempo serial aumentando linearmente (já que o teste de senha sempre será executado o máximo de vezes), enquanto o tempo paralelo permanece em uma faixa média de valores.

IV. OpenMP Tasks: Tree Reduction

A. Objetivo

O objetivo desse exercício foi paralelizar o código em C, que implementa uma redução de matrizes em forma de árvore. A versão serial do programa em `matmul-serial.c` recebe uma lista de matrizes quadradas, constrói aleatoriamente os elementos das matrizes, alternando entre os valores 1 ou -1 e retorna o resultado da multiplicação entre matrizes. Esse produto é calculado sequencialmente, com o resultado da multiplicação atual sendo armazenado e depois utilizado como parâmetro na próxima multiplicação. O cálculo da matriz resultante se baseia no algoritmo ”ingênuo”de multiplicação de matrizes, que utiliza o método mais tradicional para calcular o produto entre duas matrizes(não utiliza nenhuma otimização). Dadas duas matrizes A e B, o algoritmo utiliza o seguinte cálculo para gerar a matriz resultante C:

$$C_{i,j} = \sum_{k=0}^{m-1} A_{i,k} \cdot B_{k,j} \tag{1}$$

B. Resultados

Novamente, foi utilizada a diretiva `task` do OpenMP para implementação da versão paralelizada do código. A ideia da paralelização foi distribuir a multiplicação entre tarefas, no formato de árvore de redução. A cada iteração, o cálculo da multiplicação entre duas matrizes consecutivas da lista, é atribuído a uma thread. As threads salvam o resultado em uma lista de matrizes auxiliar, que será utilizada nas iterações seguintes, até que reste apenas a matriz resultante na lista. Caso o total de matrizes seja ímpar, uma das threads apenas copia a matriz que sobrou para a lista de matrizes. Um esboço do funcionamento da árvore de redução pode ser visualizado na Figura 4. A versão paralelizada foi implementada em `matmul-parallel.c`. O programa recebe a quantidade de matrizes que serão multiplicadas e a dimensão delas. Foram utilizadas 4 threads no bloco paralelo. Foram gerados gráficos de desempenho para análise comparativa das versões do programa.

Olhando para o gráfico de speedup por dimensão de matriz na Figura 5, vemos o aumento do speedup com o aumento de dimensão da matriz quadrada. Novamente, esse comportamento expressa a melhora no desempenho conforme aumenta a granularidade do problema. Esse comportamento é refletido no gráfico de eficiência por dimensão de matriz na Figura 6. Analisando o gráfico de tempo de execução serial e paralela por dimensão de matriz na Figura 8, vemos novamente o efeito de melhora de desempenho da versão paralelizada, quando a granularidade aumenta. Para matrizes de dimensão entre aproximadamente 2 e 25, a diferença de tempo de execução entre as versões é quase nula. Enquanto que, para matrizes com dimensão acima de 30, o tempo de execução paralela passa a ser cada vez menor, quando comparado ao tempo de execução serial. Nas Figuras 9 e 10 podemos ver os grafos representativos do tempo de execução de cada função, para a versão serial e paralela respectivamente. Podemos notar que a função que calcula efetivamente o resultado, pelo algoritmo ”ingênuo”de multiplicação de matrizes, se destaca como o maior overhead do programa em ambas as versões.

APÊNDICE

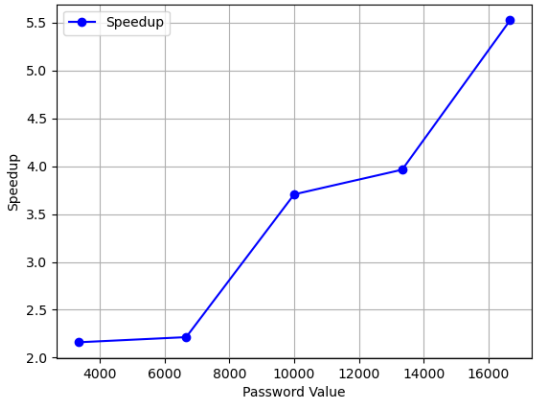


Figura 1: Gráfico de speedup por valor de senha no exercício OpenMP Tasks: ZIP

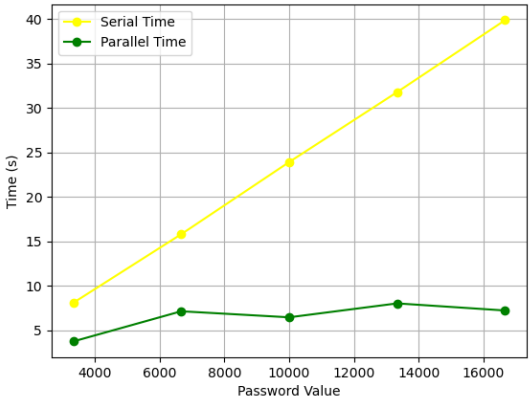


Figura 3: Gráfico de tempo de execução serial e paralela por valor de senha no exercício OpenMP Tasks: ZIP

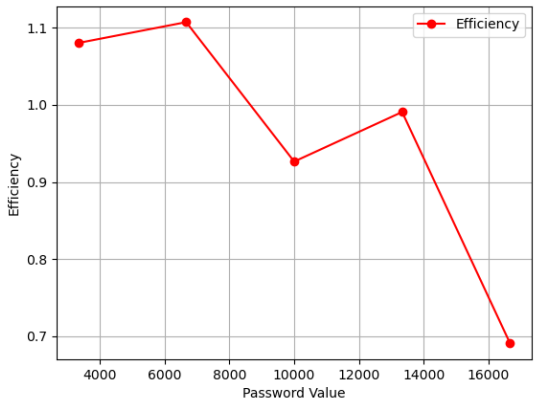


Figura 2: Gráfico de eficiência por valor de senha no exercício OpenMP Tasks: ZIP

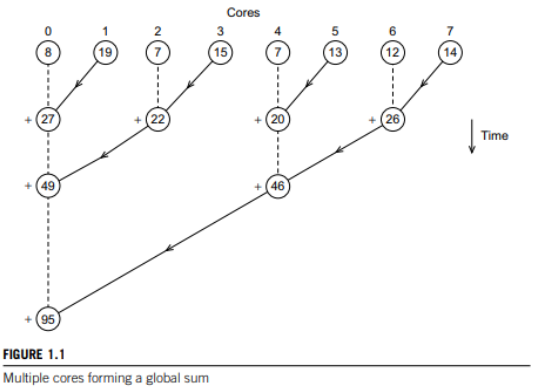


Figura 4: Exemplo de árvore de redução na multiplicação de matrizes no OpenMP Tasks: Tree Reduction

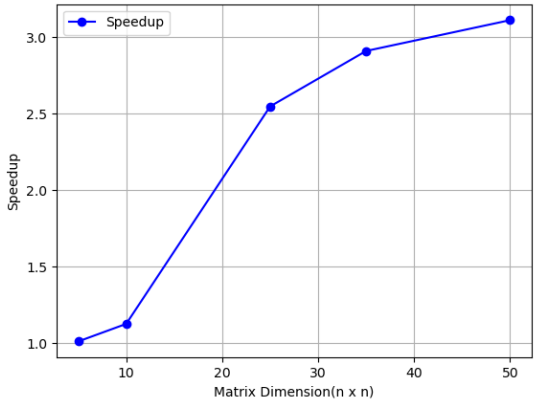


Figura 5: Gráfico de speedup por dimensão da matriz de entrada no exercício OpenMP Tasks: Tree Reduction

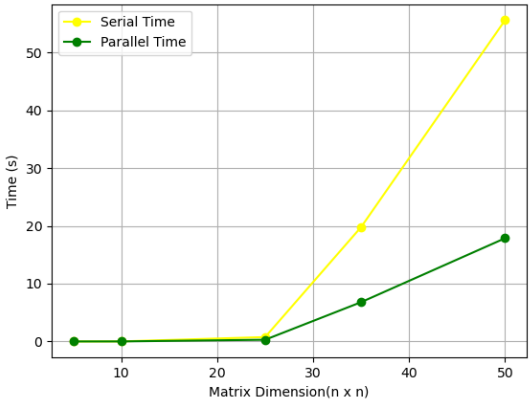


Figura 7: Gráfico de tempo de execução serial e paralela por dimensão da matriz de entrada no exercício OpenMP Tasks: Tree Reduction

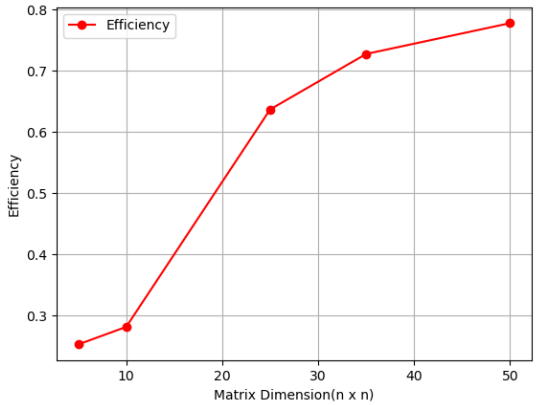


Figura 6: Gráfico de eficiência por dimensão da matriz de entrada no exercício OpenMP Tasks: Tree Reduction

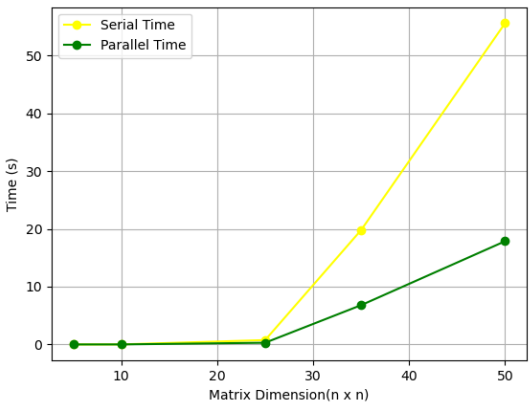


Figura 8: Gráfico de tempo de execução serial e paralela por dimensão da matriz de entrada no exercício OpenMP Tasks: Tree Reduction

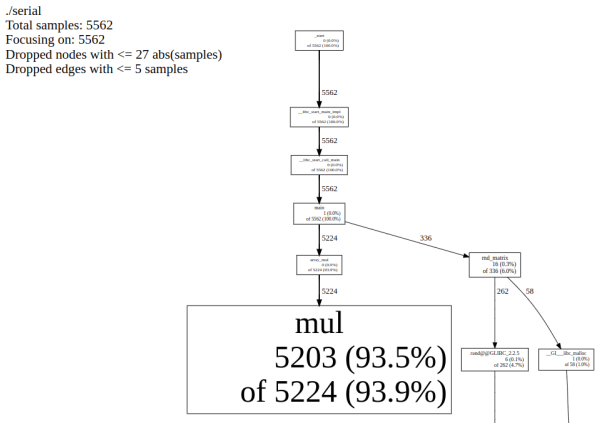


Figura 9: Grafo representativo do tempo gasto em cada função, para a versão serial no exercício OpenMP Tasks: Tree Reduction

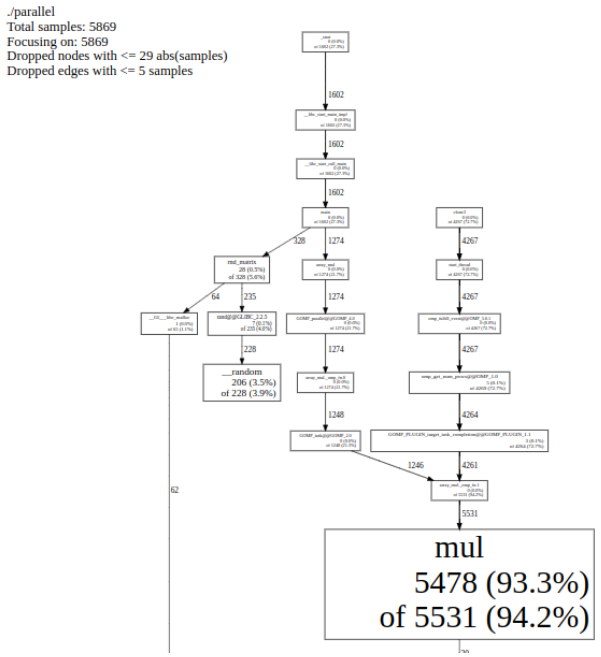


Figura 10: Grafo representativo do tempo gasto em cada função, para a versão paralela no exercício OpenMP Tasks: Tree Reduction

REFERÊNCIAS

[1] PETER, D. hyperfine. Disponível em: <https://github.com/sharkdp/hyperfine>.

[2] TIM.LEWIS. About Us. Disponível em: <https://www.openmp.org/about/about-us/>.

[3] gperftools/gperftools. Disponível em: <https://github.com/gperftools/gperftools>.

[4] RIDGERUN DEVELOPER WIKI. GPerfTools — Profiling with GPerfTools — RidgeRun. Disponível em: <https://developer.ridgerun.com/wiki/index.php/Profiling_with_GPerfTools⟩.Acessoem:19abr.2025>.

[5] AUTHOR :, O. Gperftools Heap Profiler. Disponível em: <https://gperftools.github.io/gperftools/heapprofile.html>. Acesso em: 19 abr. 2025.